

Time-Based One-Time Password TOTP

Ghadah Alharthi
Cybersecurity Department
Umm Al-Qura University
Email: s444006163@uqu.edu.sa

Atheer Alharthi
Cybersecurity Department
Umm Al-Qura University
Email: s444006152@uqu.edu.sa

Sarah Alsulami
Cybersecurity Department
Umm Al-Qura University
Email: s444002322@uqu.edu.sa

Reemas Theban
Cybersecurity Department
Umm Al-Qura University
Email: s444006605@uqu.edu.sa

Abstract—This paper presents the design and implementation of a secure two factor authentication system based on the Time-Based One-Time Password (TOTP) protocol. The system employs HMAC-SHA-1 for OTP generation, PBKDF2-SHA-256 for password hashing, and Fernet symmetric encryption for secure secret storage, providing confidentiality, authentication, replay resistance, and brute-force protection. Security evaluation confirms that replay attacks, expired OTP usage, and brute-force attempts are all effectively mitigated. The source code is available at [<https://github.com/sarah-lbrahim1/TOTP-Project>].

Keywords: Time-Based One-Time Password (TOTP), HMAC-SHA-1, Authentication, Multi-Factor Authentication (MFA)

I. INTRODUCTION

This paper presents a secure two-factor authentication system based on the Time-Based One-Time Password (TOTP) protocol.

A. Motivation

Online banking and electronic payment systems are among the most security critical environments in modern digital infrastructure, making them a primary target for adversaries. Traditional authentication relying solely on static passwords has proven inadequate: passwords can be phished, intercepted, leaked in data breaches, or reused across services, granting attackers persistent unauthorized access with no further barrier [1]. Cryptography addresses this fundamental weakness by binding each authentication event to a value that cannot be stolen and reused. TOTP achieves this by combining a shared secret key with the current timestamp through HMAC-SHA-1, producing a code that expires every 30 seconds [2]. Stored passwords are further protected through PBKDF2-SHA-256 [7], and TOTP secrets are secured at rest using Fernet symmetric encryption together forming a layered defense against both online and offline attacks.

B. Problem Statement

This work addresses the design of a secure two-factor authentication protocol for high-sensitivity environments such as online banking. Standard password-based authentication fails to meet the security demands of such systems, and even TOTP deployments frequently suffer from implementation

weaknesses that undermine their intended protections. The system must satisfy the following security requirements:

- **SR-001 Secret Key Management**

The system shall securely generate and store the shared secret key, protecting it through encryption at rest, and ensure that it is accessible only to authorized processes during OTP generation and verification. [1], [6].

- **SR-002 OTP Expiration Enforcement**

The system shall ensure that each OTP is valid only for a fixed time window of 30 seconds and reject any code outside this window [1].

- **SR-003 Replay Attack Prevention**

The system shall ensure that each OTP can be used only once and reject any attempt to reuse a previously validated code within the same validity window [1], [6].

- **SR-004 Brute Force Protection**

The system shall limit the number of failed verification attempts for both password and OTP stages, locking the account after a defined threshold [6].

- **SR-005 Standard Compliance**

The system shall implement the TOTP computation in full conformance with RFC 4226 and RFC 6238 to ensure correctness and interoperability [1], [2].

C. Contributions

- We design a two-factor authentication protocol based on TOTP that achieves replay resistance, brute-force protection, and secure secret storage.
- We implement a working prototype in Python using the standard library for the TOTP core and the cryptography library for password hashing and secret encryption.
- We provide a security analysis showing resistance to replay attacks, brute-force attacks, and offline secret extraction.

D. Our Contribution in Context

Prior studies have collectively identified the key weaknesses in TOTP deployments weak password storage, unencrypted secrets, missing replay prevention, and absent brute-force protections but most of them hasn't addressed all of them together in a single implementation. Our work fills this gap

by providing a unified prototype that integrates all of these controls simultaneously.

E. Paper Organization

The remainder of this paper is organized as follows. Section II provides background on the cryptographic primitives used and reviews related work in TOTP-based authentication. Section III presents the system design, including the threat model and detailed protocol specification. Section IV describes the implementation, covering key modules such as TOTP generation, password hashing, and secure secret storage. Section V presents functional testing and security analysis against common attack scenarios. Section VI discusses limitations and directions for future work. Section VII concludes the paper.

II. BACKGROUND AND RELATED WORK

A. Cryptographic Background

This section explains the cryptographic primitives used in the TOTP authentication system and justifies their selection.

1) *HMAC-SHA-1*: HMAC, defined in RFC 2104 [3], combines a cryptographic hash function with a secret key to produce a message authentication code:

$$\text{HMAC}(K, M) = H((K \oplus \text{opad}) \| H((K \oplus \text{ipad}) \| M)) \quad (1)$$

This two pass construction remains secure even when the underlying hash has known collision vulnerabilities [3]. Although SHA-1 is deprecated for digital signatures, it remains secure within HMAC because HMAC security depends on pseudorandomness rather than collision resistance alone [4]. RFC 6238 mandates HMAC-SHA-1 for TOTP, making it the natural choice for our implementation [1].

2) *HOTP: HMAC-Based One-Time Password*: HOTP, defined in RFC 4226 [2], generates a one-time password from a secret key K and counter C :

$$\text{HOTP}(K, C) = \text{Truncate}(\text{HMAC-SHA-1}(K, C)) \quad (2)$$

The Truncate function applies dynamic offset extraction to reduce the 20-byte HMAC output to a 6-digit code, yielding 10^6 possible values and a guessing probability of 10^{-6} per attempt [2].

3) *TOTP: Time-Based One-Time Password*: TOTP, defined in RFC 6238 [1], extends HOTP by replacing the counter with a time-derived value:

$$T = \left\lfloor \frac{\text{CurrentTime} - T_0}{X} \right\rfloor \quad (3)$$

where $X = 30$ seconds, balancing security and usability. Both client and server independently compute T , enabling verification without transmitting the code. To tolerate minor clock skew, the verification function accepts codes within a ± 1 time-step window [1].

4) *PBKDF2 for Password Hashing*: PBKDF2 [5] derives a key from a password using an iterated pseudorandom function (PRF) seeded with a random salt. The implementation uses HMAC-SHA-256 with 260,000 iterations and a per-user 16-byte salt, exceeding NIST SP 800-63B recommendations [6] and significantly increasing the cost of offline brute-force attacks.

5) *Fernet Symmetric Encryption*: Fernet provides authenticated encryption using AES-128-CBC with PKCS7 padding and HMAC-SHA-256 for integrity verification. A random initialization vector (IV) per encryption ensures distinct ciphertexts for identical plaintexts. Fernet encrypts each user's TOTP secret before storage, ensuring that a database compromise alone is insufficient to expose user secrets.

B. Related Work

The following studies examine TOTP-based authentication across different contexts and identify key security gaps.

“Implement Time Based One Time Password and Secure Hash Algorithm 1 for Security of Website Login Authentication” (2019) [7] implemented a two-factor authentication system for website login using TOTP and SHA-1. Their work addressed the vulnerability of password-only authentication to sniffing attacks, and demonstrated through Wireshark testing that a 30-second validity window is sufficient to prevent unauthorized account takeover. However, they identified that SHA-1 alone is insufficient for password protection, recommending stronger alternatives such as SHA-256, SHA-512, or AES. Our work differs by applying PBKDF2-SHA-256 with 260,000 iterations for password hashing and encrypting stored TOTP secrets using Fernet AES-128-CBC.

“An Improved Time-Based One Time Password Authentication Framework for Electronic Payments” (2020) [8] proposed a TOTP-based multi-factor authentication framework for electronic payments, combining TOTP with biometric fingerprint verification and QR code-based secret distribution over TLS. Their work addressed the weaknesses of SMS-based OTP methods, including network dependency and financial cost. However, the framework relied on password-based authentication for the first factor, which the authors acknowledged remains vulnerable to password-based attacks. Our work differs by enforcing independent brute-force lockout mechanisms for both the password and TOTP verification stages and maintaining a used-code registry to prevent replay attacks.

“Multi-Factor Authentication Using Time-based One-Time Password: A Comprehensive Analysis of Security Enhancement and Implementation Strategies” (2025) [9] conducted a comprehensive review of TOTP as a second authentication factor, analyzing its cryptographic foundations, security benefits, and implementation challenges. Their work demonstrated that MFA based on TOTP reduces account compromise by over 99% compared to password-only authentication, and identified residual vulnerabilities including improper secret key management and real-time phishing relay attacks. However, the study

remained analytical without providing a working implementation. Our work differs by providing a complete prototype that directly addresses the key management vulnerability identified in their analysis through Fernet encryption of stored secrets.

III. SYSTEM DESIGN

A. Threat Model

We define the threat model for the proposed TOTP authentication system as follows.

- **Attacker Capabilities:** The adversary is assumed to be capable of eavesdropping on all communication between the client and server, and may intercept, replay, or modify transmitted messages. The attacker may additionally possess offline access to the `users.json` storage file, enabling attempts to extract password hashes or encrypted TOTP secrets.
- **Trust Assumptions:** The system assumes that the user's device is not compromised at the time of authentication, and that the shared secret is securely delivered to the user during enrollment. System clocks are assumed to be loosely synchronized, with drift not exceeding one time step.
- **Security Goals:**
 - **Confidentiality:** The TOTP shared secret must never be exposed in plaintext, either in transit or at rest.
 - **Authentication:** Access must be granted only to users who successfully verify both a static password and a valid time-bound OTP.
 - **Replay Protection:** A previously used OTP must not be accepted again within or across authentication sessions.
 - **Brute-Force Resistance:** The system must limit successive failed authentication attempts to prevent guessing attacks against either factor.
- **Out of Scope:** The current prototype does not protect against denial-of-service attacks, network-level threats such as man-in-the-middle attacks, or compromise of the server machine itself. Secure transport and key distribution mechanisms are assumed to exist but are not implemented in this simulation.

B. Architecture Overview

The proposed TOTP authentication system is structured as a command-line simulation of a client-server authentication flow, implemented across three modules. Figure 1 illustrates the high-level architecture of the system

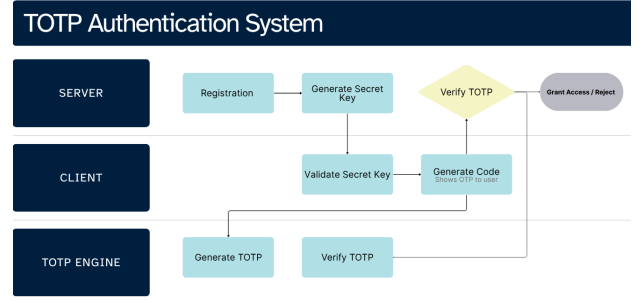


Fig. 1: System Architecture Diagram

The system consists of three logical components. The first is the Server, which constitutes the core authentication authority. It manages user registration, enforces two-factor credential verification, maintains brute-force lockout counters for both password and TOTP attempts, prevents replay attacks through a used-code registry, and decrypts stored secrets prior to verification. The second is the Client, which represents the user-side authenticator. It is responsible for accepting the shared secret, validating its format against Base32 encoding requirements, and invoking the TOTP engine to produce a time-bound 6-digit code for the user. The third is the TOTP Engine, which implements the RFC 6238 [1] algorithm in its entirety, performing time step computation, HMAC-SHA1 hashing, and dynamic truncation to generate and verify 6-digit one-time passwords.

C. Cryptographic Protocol Specification

This section presents the core TOTP authentication protocol. We describe it step by step.

1) Preliminaries: Define notation:

- K : The shared secret key, a 160-bit random value encoded in Base32
- T : The time-based counter, computed as $T = \lfloor T_{current}/X \rfloor$, where $T_{current}$ is the current Unix timestamp and $X = 30$ seconds
- $HMAC-SHA1(K, T)$: The keyed-hash message authentication code applied to counter T using secret K , producing a 20-byte digest
- $Truncate(\cdot)$: The dynamic truncation function that extracts a 4-byte string from the HMAC output and reduces it to a 6-digit OTP
- $Enc_K(m)$: Fernet symmetric encryption of message m under key K
- $H(\cdot)$: SHA-256 hash function used for username storage
- $PBKDF2(pw, salt)$: Password-based key derivation function used to hash user passwords with 260,000 iterations

2) Protocol Flow: The protocol operates across three phases.

Step 1 (Enrollment): The server generates a cryptographically random 160-bit shared secret:

$$K \leftarrow \text{Base32}(\text{os.urandom}(20)) \quad (4)$$

The secret is presented to the user once and immediately encrypted before storage:

$$\text{stored_secret} = \text{Enc}_{K_{\text{fernet}}}(K) \quad (5)$$

The user's password is hashed using PBKDF2-HMAC-SHA256 with a random salt and 260,000 iterations. The username is stored exclusively as $H(\text{username})$ to prevent plaintext identity exposure.

Step 2 (OTP Generation): The client decodes the Base32 secret and computes the time counter:

$$T = \left\lfloor \frac{T_{\text{current}}}{30} \right\rfloor \quad (6)$$

The counter is packed into an 8-byte big-endian integer and passed into HMAC-SHA1:

$$h = \text{HMAC-SHA1}(K, T) \quad (7)$$

Dynamic truncation is applied to produce the final OTP:

$$\text{OTP} = \text{Truncate}(h) \bmod 10^6 \quad (8)$$

Step 3 (Verification): Upon receiving the user-submitted credentials, the server first verifies the password using `hmac.compare_digest` to prevent timing attacks. If successful, it decrypts the stored secret and independently recomputes the expected OTP across a ± 1 time window to accommodate clock drift:

$$\text{OTP}_{\text{valid}} = \bigcup_{\delta \in \{-1, 0, 1\}} \text{Truncate}(\text{HMAC-SHA1}(K, T + \delta)) \bmod 10^6 \quad (9)$$

The submitted code is accepted only if it matches one of the three candidates and has not been previously consumed. Upon successful verification, the code is recorded in `used_codes` to prevent replay.

3) Protocol Justification:

- **Why HMAC-SHA1?** Mandated by RFC 6238 [1]. HMAC ensures the OTP is unpredictable without the secret key, even if an attacker observes many valid codes.
- **Why a 30-second time step?** RFC 6238 [1] frames this as a deliberate trade-off between security and usability, providing a sufficiently short window to limit exposure while remaining practical for users.
- **Why a ± 1 verification window?** To tolerate minor clock drift between client and server without compromising security by expanding the window further.
- **Why a 6-digit OTP?** According to RFC 4226 [2], the OTP value must be at least 6 digits to resist brute-force attacks by providing 10^6 possible values. The probability of correctly guessing a valid OTP is given by:

$$P(\text{correct guess}) = \frac{1}{10^6} \quad (10)$$

When combined with a lockout policy of $T = 3$ maximum attempts, the probability of a successful brute-force attack is bounded by:

$$P(\text{success}) \leq \frac{T}{10^6} = \frac{3}{10^6} = 0.0003\% \quad (11)$$

D. Security Analysis of Protocol

We map our protocol to the threat model defined in Section III-A.

- **Confidentiality:** The TOTP shared secret is never transmitted or stored in plaintext. It is encrypted using Fernet authenticated encryption before storage, ensuring that access to `users.json` alone is insufficient to recover any user secret.
- **Authentication:** The system enforces two-factor authentication. The first factor verifies the password using PBKDF2-HMAC-SHA256 via `hmac.compare_digest` to prevent timing attacks. The second factor verifies the time-bound OTP using HMAC-SHA1. Access is granted only upon success of both factors.
- **Replay Attack Prevention:** Each OTP is valid for a single use within its 30-second window. Successfully verified codes are recorded in `used_codes` and any reuse attempt is rejected.
- **Brute-Force Protection:** Independent failed-attempt counters are maintained for both authentication factors. After three consecutive failures at either stage, the account is locked, bounding the probability of a successful brute-force attack to $3/10^6 = 0.0003\%$.

E. Protocol Limitations

The protocol does not support secret key recovery. If a user loses their TOTP shared secret, the system provides no mechanism to retrieve or regenerate it, rendering the account permanently inaccessible.

IV. IMPLEMENTATION

A. Technology Stack

- **Language:** Python 3.13.7
- **Cryptography Library:** `cryptography` used for Fernet symmetric encryption and PBKDF2 password hashing
- **Standard Library Modules:** `hmac`, `hashlib`, `struct`, `base64`, `time`, `os`, `json` used for HMAC-SHA1 computation, secret generation, and persistent user storage
- **Version Control:** GitHub

B. Key Implementation Components

1) **TOTP Generation:** Listing 1 shows the core OTP generation implementation. The function accepts the Base32-encoded shared secret, decodes it into raw bytes, computes the time-based counter, and applies HMAC-SHA1 followed by dynamic truncation to produce a 6-digit OTP.

```
1 def generate_totp(secret, interval=30, digits=6):
2     key = base64.b32decode(secret)
3     counter = int(time.time() / interval)
4     counter_bytes = struct.pack(">Q", counter)
5     hmac_hash = hmac.new(key, counter_bytes,
6                           hashlib.shal).digest()
7     offset = hmac_hash[-1] & 0x0F
8     code = struct.unpack(">I",
9                           hmac_hash[offset:offset+4])[0] & 0
10    x7fffffff
```

```

10 otp = code % (10 ** digits)
11 return str(otp).zfill(digits)

```

Listing 1: TOTP Generation

2) *Password Hashing and User Registration:* Listing 2 Each password is hashed using PBKDF2-HMAC-SHA256 with a randomly generated salt and 260,000 iterations. The TOTP secret is encrypted using Fernet prior to storage, and the username is persisted exclusively as a SHA-256 digest.

```

1 def _hash_password(password):
2     salt = os.urandom(16).hex()
3     hashed = hashlib.pbkdf2_hmac(
4         "sha256", password.encode(),
5         salt.encode(), 260000).hex()
6     return f"{salt}${hashed}"
7
8 def register_user(username, password):
9     secret = base64.b32encode(os.urandom(20)).decode()
10
11     hashed_username = _hash(username)
12     hashed_password = _hash_password(password)
13     encrypted_secret = fernet.encrypt(
14         secret.encode()).decode()
15
16     users[hashed_username] = {
17         "password": hashed_password,
18         "secret": encrypted_secret
19     }

```

Listing 2: Password Hashing and User Registration

3) *Replay Attack Prevention and Brute-Force Lockout:* Listing 3 shows the verification logic for replay prevention and brute-force lockout. Successfully verified codes are recorded in `used_codes` indexed by username and counter value. Independent failed-attempt counters are maintained for both the password and TOTP stages, locking the account after three consecutive failures.

```

1 window_keys = {
2     f"{hashed_username}:{current_counter+delta}:{code}"
3     for delta in range(-WINDOW, WINDOW + 1)
4 }
5 already_used = bool(window_keys & used_codes)
6
7 if verify_totp(secret, code):
8     if already_used:
9         print("Code already used. Authentication
10             FAILED")
11     else:
12         used_codes.update(window_keys)
13         print("Authentication SUCCESS")
14 else:
15     totp_info["count"] += 1
16     if totp_info["count"] >= MAX_ATTEMPTS:
17         totp_info["locked"] = True
18         print("Account is now LOCKED")

```

Listing 3: Replay Prevention and Brute-Force Lockout

C. Simulation Environment

The system mimics a client-server authentication scenario using a menu-based command-line interface that runs entirely on a single host computer. These include the following three stages of the protocol: enrollment, OTP creation, and OTP validation. The enrollment stage entails generating a cryptographically secure shared secret on the server side, encrypting

it, and storing the user record in `users.json`. The OTP creation stage involves receiving the shared secret on the client side and using the TOTP engine to generate the current 6-digit one-time password. OTP validation involves loading credentials from the persistent layer, authenticating using two factors, and implementing replay attack and brute-force attack mitigation strategies.

D. Implementation Challenges and Solutions

- **Replay Attack Prevention Across the Verification Window:** Implementing replay prevention while simultaneously supporting a ± 1 time window introduced a conflict: the system must accept codes from adjacent time steps to tolerate clock drift, yet reject any code that has already been used within that window.
Solution: Each consumed code is recorded in `used_codes` as a composite key combining the username hash, counter value, and code string. This ensures that a code is rejected upon reuse regardless of which time step it was valid for, without restricting legitimate use of adjacent window codes.
- **Secure Secret Storage Without Exposing the Encryption Key:** Storing the TOTP secret in encrypted form required managing a symmetric encryption key that must persist across sessions without being embedded in the source code or stored alongside the encrypted data.
Solution: The encryption key is generated once using `Fernet.generate_key()` and persisted in a separate `secret.key` file managed exclusively by `config.py`. This isolates key management from credential storage, ensuring that `users.json` and `secret.key` must both be compromised for an attacker to recover any TOTP secret.

V. EVALUATION

A. Functionality Testing

We verified that the implemented TOTP authentication system correctly satisfies the defined functional requirements.

Test Case 1: Successful Authentication

- **Input:** Valid username, password, and TOTP code generated using the shared secret key.
- **Expected:** User is authenticated successfully and granted access.
- **Result:** Passed, the system verified both credentials and the TOTP code, then granted access.

Test Case 2: Invalid TOTP

- **Input:** Valid username and password with an incorrect TOTP code.
- **Expected:** Authentication fails and access is denied.
- **Result:** Passed, the system rejected the invalid OTP and denied access.

Test Case 3: Expired TOTP

- **Input:** Valid username and password with a TOTP code submitted after its validity period.
- **Expected:** Authentication failure due to expiration.

- **Result:** Passed. The system correctly detected the expired OTP and rejected the authentication request.

Table I summarizes all test results.

TABLE I: Functionality Test Results

Test Case	Expected Outcome	Actual Outcome	Status
Successful Authentication	Access granted	Access granted	Pass
Invalid TOTP	Authentication rejected	Rejected	Pass
Expired TOTP	Authentication rejected	Rejected	Pass

B. Security Testing

The security evaluation was conducted to assess the system's resistance to common TOTP attacks.

1) *Replay Attack:* **Attack:** An attacker captures a valid OTP and attempts to reuse it after successful authentication.

Our Implementation: The system enforces OTP expiration and tracks previously validated OTP values within the authentication session.

Result: The attack was successfully prevented, and reused OTPs were rejected.

2) *Expired OTP Attack:* **Attack:** An attacker attempts to authenticate using an OTP after the 30-second validity window has expired.

Our Implementation: OTP verification is based on the current time step and rejects codes outside the accepted time window.

Result: Expired OTPs were consistently rejected.

3) *Brute-Force Attack:* **Attack:** An attacker repeatedly submits randomly generated OTP values in an attempt to guess a valid code.

Our Implementation: Multiple incorrect OTP values were entered consecutively during authentication.

Result: Attack Failed.

Analysis: The system limits failed authentication attempts and temporarily blocks further verification requests after reaching the threshold. Combined with the 6-digit OTP space and short validity period, this significantly reduces the probability of a successful brute-force attack.

VI. DISCUSSION

A. What Worked Well

- The system successfully generated and verified TOTP codes.
- Resistance was observed against replay, expired OTP, and brute-force attacks.
- Legitimate users were authenticated correctly.
- OTP validity was strictly enforced within a 30-second window.

B. What Could Be Improved

- The prototype could be extended to a web or mobile platform.
- Additional security testing scenarios could be included.
- Real-world deployment testing would improve validation.

C. Real-World Deployment Challenges

If this system were deployed in production, we would need to address:

- **Key Management:** Protecting and securely storing cryptographic secrets.
- **Scalability:** The ability of the system to efficiently support a growing number of users and authentication requests.
- **User Experience:** Balancing security requirements with ease of use.
- **Regulatory/Compliance:** Following applicable cybersecurity and data protection regulations.

VII. CONCLUSION AND FUTURE WORK

A. Conclusion

We designed and implemented a TOTP-based authentication system for secure user login scenarios. Our system provides time-based one-time password generation and verification, improving overall authentication security. The implementation demonstrates a successful and efficient two-factor authentication mechanism. Security analysis shows resistance to replay attacks, expired OTP usage, and brute-force attempts, though limitations remain in key management, scalability in large-scale deployments, and usability issues related to time synchronization.

B. Future Work

Several directions remain for future work:

- **Web and mobile platform support:** The system could be extended to web and mobile applications using secure APIs with encryption for consistent and secure TOTP authentication across devices.
- **Improved security using behavior analysis:** The system could be enhanced by analyzing user behavior such as login time, location, and device information to detect unusual activity and apply additional verification when needed.
- **Cloud deployment and scalability:** The system could be deployed in a cloud environment with scalable services and secure key management to support a growing number of users.

ACKNOWLEDGMENT

The authors thank Dr. Shorouq Alansari for guidance and feedback throughout the project.

REFERENCES

- [1] M'Raihi, D., Machani, S., Pei, M., & Rydell, J. (2011). *TOTP: Time-based one-time password algorithm* (RFC 6238).
- [2] IETF. (2005). *HOTP: An HMAC-Based One-Time Password Algorithm* (RFC 4226). <https://datatracker.ietf.org/doc/html/rfc4226>
- [3] Krawczyk, H., Bellare, M., & Canetti, R. (1997). *HMAC: Keyed-Hashing for Message Authentication* (RFC 2104). IETF. <https://datatracker.ietf.org/doc/html/rfc2104>
- [4] Bellare, M. (2006). New proofs for NMAC and HMAC: Security without collision-resistance. In *Advances in Cryptology – CRYPTO 2006*, pp. 602–619. Springer.
- [5] Turan, M., Barker, E., Burr, W., & Chen, L. (2010). *Recommendation for Password-Based Key Derivation – Part 1: Storage Applications* (NIST SP 800-132). National Institute of Standards and Technology.
- [6] Grassi, P., Garcia, M., & Fenton, J. (2017). *Digital Identity Guidelines: Authentication and Lifecycle Management* (NIST SP 800-63B). National Institute of Standards and Technology.
- [7] H. Seta, T. Wati, and I. C. Kusuma, "Implement Time Based One Time Password and Secure Hash Algorithm 1 for Security of Website Login Authentication," in *Proc. 2019 International Conference on Informatics, Multimedia, Cyber and Information System (ICIMCIS)*, Jakarta, Indonesia, 2019, pp. 115–120.
- [8] M. A. Hassan, Z. Shukur, and M. K. Hasan, "An Improved Time-Based One Time Password Authentication Framework for Electronic Payments," *International Journal of Advanced Computer Science and Applications (IJACSA)*, vol. 11, no. 11, pp. 359–366, 2020.
- [9] S. Mishra, K. Meenakshi, and D. Bhargava, "Multi-Factor Authentication Using Time-based One-Time Password: A Comprehensive Analysis of Security Enhancement and Implementation Strategies," in *Proc. 2025 Modern Electronics Devices and Intelligent Communication Systems (MEDCOM)*, 2025, pp. 100–105.

APPENDIX

A. Expected Output

The following output demonstrates the three protocol stages. First, a user registers and receives a Base32-encoded shared secret.

```
----- TOTP SYSTEM -----
1 Register user
2 Generate TOTP
3 Login
4 Exit
Choose option: 1
Username: sarah
Password: sarah
User registered
Secret: JUPPYUQDIZIUHJCP75VTYSM5W5JNBWQA
```

Fig. 2: User registration and secret generation.

The secret is encrypted via Fernet and stored in `users.json`; the plaintext is never written to disk.

The user enters the shared secret into the client, which generates a 6-digit time-bound OTP.

```
----- TOTP SYSTEM -----
1 Register user
2 Generate TOTP
3 Login
4 Exit
Choose option: 2
Enter secret: JUPPYUQDIZIUHJCP75VTYSM5W5JNBWQA
Your TOTP code: 660506
```

Fig. 3: TOTP code generation on the client side.

The code is valid for 30 seconds and changes with each time step. The user authenticates successfully, then immediately resubmits the same code to test replay prevention.

```
----- TOTP SYSTEM -----
1 Register user
2 Generate TOTP
3 Login
4 Exit
Choose option: 3
Username: sarah
Password: sarah
Enter TOTP code: 660506
Authentication SUCCESS

----- TOTP SYSTEM -----
1 Register user
2 Generate TOTP
3 Login
4 Exit
Choose option: 3
Username: sarah
Password: sarah
Enter TOTP code: 660506
Code already used. Authentication FAILED
```

Fig. 4: Successful authentication followed by replay attack rejection.

The system grants access on the first attempt and rejects the second with Code already used. Authentication FAILED, confirming the used-code registry operates correctly.

TEAM CONTRIBUTIONS

- **Ghadah Alharthi:** System design, threat model, cryptographic protocol specification, and security analysis.
- **Atheer Alharthi:** Introduction, background on cryptographic primitives (HMAC-SHA-1, HOTP, TOTP), and related work review.
- **Sarah Alsulami:** Implementation of TOTP generation, password hashing, secure secret storage, and code documentation.
- **Reemas Theban:** Evaluation and security testing, functionality test cases, discussion, conclusion, and paper writing.

REPOSITORY INFORMATION

<https://github.com/sarah-Ibrahim1/TOTP-Project>